

- 1. Why did Fibonacci fail the testThrowsIllegalArgumentException test? What did you have to do to fix it?**
 - a. The test failed because getFibTerm(int n) originally threw an exception whenever n was less than or equal to zero because it was being interpreted as a negative number, when it actually isn't. To fix this, I changed the if statement to only throw an exception when n is less than zero. I then created a new else if statement for when n is zero.
- 2. Why did Fibonacci fail the testBaseCase test? What (if anything) did you have to do to fix it?**
 - a. The test failed because calling getFibTerm(0) did not return 0, and calling getFibTerm(1) did not return 1. To fix this, I used the newly created else if statement for when n is zero stated in question 1, I had the program return 0. Then, I changed the else if (n<2) to else if (n == 1 || n==2) and had the program return 1.
- 3. Why did Fibonacci fail the testInductiveCase test? What (if anything) did you have to do to fix it?**
 - a. This test failed because the final else case would return a number calculated from 2 recursive calls for the function. However, the number was incorrect, so I changed it to make the program return (getFibTerm(n-1) + getFibTerm(n-2)).
- 4. Why did Fibonacci fail the testLargeN test? What (if anything) did you have to do to fix it?**
 - a. This test failed because the number returned was incorrect. This happened due to integer overflow, as our resulting number was larger than an integer can hold. To fix this, I changed the return type to "long" from "int". While this fixed integer overflow, it resulted in a seemingly forever running test case, which brings me to question 5.
- 5. What was causing Fibonacci to be so slow on testLargeN test? What did you do to make Fibonacci faster while still preserving the recursive nature of your implementation?**
 - a. The reason Fibonacci was so slow was because of the nature of the program design. While the Fibonacci design I created based on the given code was correct, it would require a tree to be built with lots of branches. Once the branches were finished, the program would have to travel up all the branches again to find the answer and return it. This was very inefficient. To preserve the recursive nature of my implementation, I created an array list inside Fibonacci which would store the resulting fibonacci number at the index of its call number (or n in this case). I first check if n<0 and throw an exception if it is. Otherwise, we move to checking if the size of the array list is greater than n. If it is, we return the number at index n of the arraylist. Otherwise, we go through our base cases, and in each add 0 or 1 to the array list. Next, our final else statement now adds the result from the formula (getFibTerm(n-1) + getFibTerm(n-2)) to the list and then returns the number at index n. Since the program stores any results it gets for the tree in the arrayList, numbers can be "skipped" and be located

immediately, significantly decreasing program runtime. To stay recursive, we simply use recursive calls to find the number to add to the array list.